# COMPASS: mapping where life-like behaviour appears in stochastic models, with an ordered, non-repeating active search

**Draft 1, for review** · Gokyona Wahiro (pen name), independent researcher, Toronto · 29 October 2026

Code and data: https://github.com/Gokyona/life-origins-traversal · Text: CC BY 4.0 · Code: MIT · Outline: method_outline.md · Detailed reports: stage 1, stage 2, stage 3, stage 4a · Companion paper: Stage 4 environment search

> **Status of this draft.** This is the full (arXiv) version. The ALIFE version will keep Sections 1–5 and shorten 6–7. Every number is taken from the stage reports linked in each section; it will be re-checked against the result files before submission, as was done for the companion paper.

---

## Abstract

Models of the origin of life are stochastic, slow and depend on many conditions. The useful question is often not what one run does, but where in condition space a level of life-like behaviour appears at all. That question asks for a boundary. We present COMPASS (Condition-space Ordered Mapping by Prioritised Active Sampling of Stochastic systems). It tests conditions in an ordered, never-repeating sequence, simulates each condition once, fits a shared neural map of the success probability, and spends most of its budget where a bootstrap ensemble of maps disagrees. None of the parts is new. Our contribution is the combination for this question, a benchmark protocol in which all methods share one map-maker and are scored against brute-force ground truth, and a record of what did not work.

On six models, from a toy protocell to a fully bottom-up chemistry with 29 dimensions, COMPASS needed 1.6–8× fewer simulations than Sobol or random sampling for the same map accuracy. The saving held at 20–30 dimensions (2.8–4.0×), and the engine's own cost did not grow with dimension. On autocatalytic sets, the threshold read off a COMPASS map lay within 7% of direct simulation. Ablations show that the saving comes from the loop as a whole: a standard boundary rule (straddle) does as well as our scoring. On a 54-dimensional bottom-up model, COMPASS found and confirmed environments for levels that a ground-truth sample never hit. But it does not raise hits per simulation uniformly, and most of its apparent advantage in raw hit counts comes from the design of the comparison.

## 1. Introduction

Simulations of the origin of life usually pick one set of conditions, run a model for a long time and wait for something life-like. A negative run is then hard to read. The conditions may be wrong, or the run too short, or the event simply rare. We take a different view: "does life emerge here?" is a question about a map. A model's condition space splits into a region where a level of life appears with probability above one half, and a region where it does not. We want the boundary between them.

Brute force is expensive. A model with 10 conditions and a run time of milliseconds can be mapped on a grid, but one with 30 or 50 conditions and runs of seconds cannot. Most of a uniform sample falls far from the boundary, where the answer is obvious. The boundary itself is noisy, because each run is random.

Active learning is built for this. Query by committee (Seung, Opper & Sompolinsky 1992), Bayesian active learning by disagreement (Houlsby et al. 2011) and level-set estimation (Bryan et al. 2005; Gotovos et al. 2013) all spend samples where the model is uncertain (see Settles 2009 for a survey). Deep ensembles give cheap

uncertainty for neural maps (Lakshminarayanan, Pritzel & Blundell 2017). Quasi-random sequences give even coverage (Sobol' 1967; Owen 1998). We combine these parts into one loop for a specific use: mapping levels of life in stochastic models on a single GPU. We test it on models where the answer is known, against matched baselines, and report negative results along with positive ones.

## 2. The COMPASS loop

**Problem.** A model has d conditions, scaled to the unit cube. One simulation at a condition u returns a binary outcome, so each condition has a success probability p(u). The goal is to classify conditions as life ($p > 0.5$) or not, with as few simulations as possible.

**Components.**

- **R1, never repeat.** Each tested condition is quantised to a canonical key (4,096 levels per dimension) and archived. A condition whose key is already in the archive is never simulated again.
- **R2, ordered coverage.** Candidates come from one scrambled Sobol sequence of $2^{19}$ points. The engine starts with the first 2,048, and in every later round 30% of the batch continues along the sequence.
- **R3, one run per condition.** Repeated runs per condition were consistently wasteful: the map-maker averages neighbouring single runs better than repetition does.
- **R4, a shared map-maker.** A small neural network (two hidden layers of 256 units) is fitted to all outcomes with binary cross-entropy, with early stopping on a 10% held-out split.
- **R5, refinement by disagreement.** Five networks, each trained on a bootstrap resample, score every untested candidate by the spread of their predictions. Spread is high where more data would change the map, and low where the outcome is simply random.
- **R6, soft selection.** 70% of each batch is drawn with probability proportional to the squared spread, which spreads points along the boundary.
- **R7, growing batches.** Each round adds max(1,024, N/3) points, so the number of rounds grows only logarithmically with the budget. The five networks are trained together as one batched network.
- **R8, a joint map for nested levels** (from stage 3). When levels are nested (a higher level implies the lower ones), one network predicts all levels with a nesting constraint.

```
pool <- scrambled Sobol sequence (2^19 points); archive <- {}
simulate the first 2048 pool points once each
while runs < budget:
    batch <- max(1024, runs / 3)
    30% of batch: next untested pool points in sequence order
    fit 5 bootstrap networks on all (u, outcome)
    score untested pool points by the spread of the 5 predictions
    70% of batch: draw with probability ~ score^2
    simulate the new points once each; archive their keys
final map <- early-stopped network on all data
```

For stage 4, environments outside a region's constraints are masked: they stay in the sequence but are never simulated.

## 3. Benchmark protocol

- **Ground truth.** 4,000–8,000 held-out conditions, each simulated 64–128 times by brute force (stage 4: 500–2,000 conditions with 16–32 seeds).
- **Methods.** Uniform random and Sobol sampling with one run per condition, and COMPASS variants.
- **One map-maker for all.** Each method produces an ordered list of simulated conditions. At fixed budgets, the same map-maker (R4) is fitted to the prefix of each list and scored on the held-out set. Methods therefore differ only in which conditions they chose.
- **Hard set.** Accuracy is also reported on conditions with $0.1 \le p \le 0.4$ or $0.6 \le p \le 0.9$: random, but still decidable.
- **Repetitions.** 3–5 per method for the test models; 1 for the stage 4 maps (2 for one focused map in 4c-3).
- **Hardware.** One Intel Arc Pro B70 (32 GB), PyTorch on the XPU backend, a 6-core desktop CPU.

## 4. Test models

| Model | Conditions | Level mapped | Notes | Source |
|---|---|---|---|---|
| Protocell chemostat | 10 | self-replication above background | Poisson tau-leaping; Eigen error threshold built in; 2 distractors | stage 1 |
| Autocatalytic sets | 7 | a RAF set exists | binary polymer model (Hordijk & Steel 2004), exact maxRAF on the GPU; 2 distractors | stage 1e |
| Stochastic corrector | 8 | heredity through division (L2) | 128 cells, after Szathmáry & Demeter (1987); 2 distractors | stage 2a |
| Coupled protocell | 10 | protocell in homeostasis (L3) | self-made membrane triggers division, after Gánti (2003); 2 distractors | stage 2b |
| Coupled protocell, padded | 20 and 30 | L3 | extra dimensions either ignored (distractors) or each with a small real effect (weak) | stage 4a |
| Bottom-up chemistry, layer 1 | 29 | activated feedstock (E1), persistent chains (E2) | 19 environmental and 10 unknown-constant dimensions | stage 4c-1 |

## 5. Results

### 5.1 Fewer simulations for the same map

| Model | Saving against Sobol (or random) at equal accuracy | Evidence |
|---|---|---|
| Protocell chemostat | about 8× | COMPASS at 32k matches Sobol at 256k: 0.988–0.989 against 0.989 on all conditions, 0.938–0.942 against 0.942 on the hard set |
| Autocatalytic sets | about 2× | |
| Stochastic corrector, L2 | about 8×, and 40–50% fewer errors at equal budget | |
| Coupled protocell, L3 | about 4× | all active rules reach at 16k the accuracy Sobol reaches at 64k |
| Padded to 20–30 dimensions | 3.6–4.0× with distractors, 2.8–2.9× with weak dimensions (4.4× at 10) | |
| Bottom-up chemistry, E1 / E2 | 2× / 1.6× to reach Sobol's final accuracy; 2.4× / 2.9× to reach 95% of the achievable gain over a trivial classifier | see below |

The saving depends on the boundary. It is largest where the boundary is thin and the life region small (the chemostat and the stochastic corrector), and smallest where a large part of the space is plainly negative (the bottom-up chemistry).

**Bottom-up chemistry (stage 4c-1).** This is the only stage 4 layer with a matched Sobol arm (one repetition each; accuracy on ground-truth test points).

| Level | Method | 2k | 4k | 8k | 16k | 32k | 64k | Runs to 95% of the gain |
|---|---|---|---|---|---|---|---|---|
| E1 (trivial 0.559) | Sobol | 0.889 | 0.933 | 0.939 | 0.948 | 0.965 | 0.963 | 15,515 |
| | COMPASS | 0.892 | 0.928 | 0.955 | 0.963 | 0.966 | 0.968 | 6,531 |
| E2 (trivial 0.842) | Sobol | 0.935 | 0.942 | 0.968 | 0.968 | 0.972 | 0.978 | 36,336 |
| | COMPASS | 0.934 | 0.956 | 0.969 | 0.976 | 0.977 | 0.980 | 12,387 |

"Trivial" is the accuracy of always predicting the majority class. At 90% of the gain the two methods need about the same number of runs, so the advantage appears in the last part of the map, near the boundary.

### 5.2 Scaling with dimension

Padding the coupled protocell model to 20 and 30 dimensions (3 repetitions each, budget 64k) kept COMPASS ahead of Sobol at every budget from 4k on. Accuracy at 64k on all conditions was 0.980 against 0.967 (D = 20, distractors) and 0.980 against 0.971 (D = 20, weak). On the hard set it was 0.707 against 0.617 and 0.722 against 0.652. The engine's cost stayed at about 75 s per 64,000-simulation run, independent of dimension, so no screening step was needed before the 29–54-dimensional stage 4 maps.

### 5.3 Maps agree with theory

On autocatalytic sets, the GPU simulator first reproduced the published catalysis threshold for RAF sets (Hordijk, Kauffman & Steel 2011) to within 2% for maximum polymer length n = 7–10. The threshold read off

a COMPASS map built from 32,000 simulations lay within 7% of the directly simulated value on average, across 15 slices of the condition space. In the bottom-up chemistry, the same threshold re-emerged from the catalysis layer: P(RAF) = 0.5 at 1.05 (n = 6) and 1.27 (n = 8) catalysed reactions per molecule, close to the test model's direct values (about 1.05–1.1 and 1.25).

### 5.4 What matters in the engine

**The loop, not the formula** (stage 3e; coupled protocell at L3, 5 repetitions).

| Selection rule inside the same loop | 4k | 16k | 64k |
|---|---|---|---|
| Sobol, no selection | 0.953 ± 0.002 | 0.961 ± 0.002 | 0.976 ± 0.002 |
| Disagreement, top-k | 0.956 ± 0.002 | 0.975 ± 0.001 | 0.986 ± 0.001 |
| Disagreement², sampled (default) | 0.959 ± 0.002 | 0.977 ± 0.003 | 0.987 ± 0.001 |
| Straddle, top-k (Bryan et al. 2005) | 0.957 ± 0.003 | 0.975 ± 0.001 | 0.987 ± 0.001 |
| Straddle², sampled | 0.959 ± 0.001 | 0.976 ± 0.000 | 0.987 ± 0.001 |

All four active rules reach at 16k what Sobol reaches at 64k. On the hard set no rule leads by more than one standard deviation. The saving therefore comes from the loop: an ensemble map-maker, growing rounds, a fixed exploration share and no repeated conditions.

**A joint map for nested levels.** On a four-level model, separate maps predicted a higher level without a lower one at 1.1–1.4% of test points. A joint map removed all such contradictions, at the same accuracy and with 3× shorter training (10 s against 30 s at 32k).

**Soft selection and batched training** (stage 1d). Soft selection added 1.9 and 2.5 points on the hard set at 4k and 8k. Training the five networks as one batched network cut the engine's time per repetition from about 510 s to about 128 s.

### 5.5 Negative results

- **The first engine lost to random search** (stage 1). It used repeated runs with a sequential test, a nearest-neighbour map and refinement by $\hat{p}(1 - \hat{p})$. The refining engines were below plain Sobol sampling at almost every budget, with a dip to 0.68 on the hard set at 4k runs. One run per condition beat every multi-run method from 2k to 16k runs; repeated runs paid off only at the largest budget on the hard set (Sobol with 8 runs 0.907 against 0.892 with one, at 128k). Two lessons shaped COMPASS: spend the budget on more conditions rather than repeats, and do not score by $p(1 - p)$, which chases conditions that are random by nature.
- **A first apparent win was an artefact.** With a map-maker trained for a fixed number of steps, COMPASS seemed to lead by 8–12 points on the hard set at every budget from 4k on. The baselines were underfitted. With early stopping every method improved (random at 256k: hard set 0.702 → 0.944), and the lead became a saving in simulations instead of a permanent gap.
- **Cheap screening helps little** (stage 2d). A simulator 7.8× cheaper agreed with the full one on 97.2% of conditions, but only 60% near the boundary, where it was optimistic. Screening with it helped only up to 8k runs (by up to 0.006); from 16k on, no variant beat COMPASS on full simulations alone.

## 6. Use on a 54-dimensional bottom-up model

The companion paper uses COMPASS to map a bottom-up chemistry over 13 environmental variables (19 dimensions) and up to 35 unknown constants. There, the rarest levels (heredity with long copies, and heredity inside vesicles) never appeared in ground-truth samples of 8,000–16,000 runs. COMPASS found environments for both and confirmed each with 16 new seeds.

Is that a property of the search? We checked in two ways.

**Equal budget.** COMPASS runs were truncated to the same number of feasible runs as the ground truth in each map (38,672 in total). COMPASS found heredity with long copies 4 times and the joint level once, and the ground truth neither. If both had the same rate, all 4 hits would fall in COMPASS with probability 1/16 (one-sided $p \approx 0.06$). This is suggestive, not significant.

**Distinct hits against a single-seed random sampler.** The ground truth repeats each point 16 times, so it tests far fewer distinct environments than COMPASS at the same budget. We therefore compared COMPASS with the expected number of distinct hits of a uniform random sampler with one seed per point (the ground-truth rate per run times the budget).

| Map, level | Ground truth (16 seeds) | Random, 1 seed (expected) | COMPASS |
|---|---|---|---|
| map4c3, heredity (strict) | 9 | 15 | **32** |
| map4c4, heredity (strict) | 10 | 24 | 27 |
| box3 v2, filled vesicles (strict) | 13 | 163–208 | 220 |
| box2, catalytic network | 2 | 19–23 | 4–6 |
| map4c4, filled vesicles (strict) | 2 | 32 | 16 |

- **Most of the raw advantage over the ground truth is the 16-seed design.** For filled vesicles in box3 v2, COMPASS found 220 distinct points against 13 for the ground truth, but about 208 would be expected from a single-seed random sampler.
- **COMPASS is better for some levels and worse for others.** It found twice as many heredity points as random in map4c3, but fewer catalytic networks in box2 and fewer filled vesicles in map4c4. There it concentrates on its uncertain boundary, not on hits.
- **The one gain random sampling cannot match** is reaching levels whose random rate is too small to measure (below 1 in 16,000 runs).

So COMPASS is a map-maker, not a hit-finder. When the goal is to find rare positives, a selection rule aimed at hits would likely do better.

## 7. Limits

- **The saving depends on the problem.** It is 1.6–8× in our models and smallest in mostly negative spaces.
- **Engine overhead.** Map training costs about 72–78 s per 64,000-simulation run in the current engine (stages 3e and 4a), and about 85–100 s in the earlier stage 1–2 versions. When one simulation takes microseconds, this dominates; for stage 4, where a run takes from 45 ms (4c-1) to about 2.6 s (worst case in 4c-4), it is small.

- **Baselines.** We compare against random and Sobol sampling at full scale, and against straddle in one model. We have not compared with Gaussian-process level-set methods (Gotovos et al. 2013), which scale poorly to tens of thousands of points but may do well at small budgets.
- **Repetitions.** Test models have 3–5 repetitions; the stage 4 maps mostly have one, and only one has a matched Sobol arm (stage 4c-1).
- **Rare levels.** Accuracy curves cannot measure maps of very rare levels: in stage 4, always predicting "no" was already 97–100% accurate for them at every budget. For those levels we rely on hit counts, which have the caveats of Section 6.

## 8. Code, data and use of AI

All code, specifications, result files with md5 sums, and failed tests are in the repository. Most raw data from the stage 4 maps are on the author's computer, and will be uploaded before submission with checksums.

The author used an AI assistant, Claude (Anthropic), throughout:

- **Engine and benchmarks:** it designed most of the engine and wrote all code.
- **Running and checking:** it ran the benchmarks on the author's computer, under rules set by the author, and diagnosed the failures reported in Section 5.5.
- **Writing:** it drafted the reports and this paper.

The author set the research question and the requirement that the search be ordered and never repeat a condition. The author made the main decisions and is responsible for every claim.

## References

Each entry was checked against the publisher's page, the authors' copy or the proceedings page. The check notes are in the stage 1, stage 2 and stage 3 papers.


- Bryan, B., Nichol, R. C., Genovese, C. R., Schneider, J., Miller, C. J. & Wasserman, L. (2005). Active learning for identifying function threshold boundaries. *Advances in Neural Information Processing Systems* 18, 163–170.
- Gánti, T. (2003). *The Principles of Life.* Oxford University Press.
- Gotovos, A., Casati, N., Hitz, G. & Krause, A. (2013). Active learning for level set estimation. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, 1344–1349.
- Hordijk, W. & Steel, M. (2004). Detecting autocatalytic, self-sustaining sets in chemical reaction systems. *Journal of Theoretical Biology* 227(4), 451–461.
- Hordijk, W., Kauffman, S. A. & Steel, M. (2011). Required levels of catalysis for emergence of autocatalytic sets in models of chemical reaction systems. *International Journal of Molecular Sciences* 12(5), 3085–3101.
- Houlsby, N., Huszár, F., Ghahramani, Z. & Lengyel, M. (2011). Bayesian active learning for classification and preference learning. arXiv:1112.5745.
- Lakshminarayanan, B., Pritzel, A. & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems* 30.
- Owen, A. B. (1998). Scrambling Sobol' and Niederreiter–Xing points. *Journal of Complexity* 14(4), 466–489.

- Settles, B. (2009). Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison.
- Seung, H. S., Opper, M. & Sompolinsky, H. (1992). Query by committee. In *Proceedings of the 5th Annual Workshop on Computational Learning Theory (COLT)*, 287–294.
- Sobol', I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics* 7(4), 86–112.
- Szathmáry, E. & Demeter, L. (1987). Group selection of early replicators and the origin of life. *Journal of Theoretical Biology* 128, 463–486.